Universidade Federal do Maranhão – UFMA
Centro de Ciências Exatas e Tecnologia - CCET
Departamento de Engenharia Elétrica
Av. dos Portugueses, no 1966, Cidade Universitária Dom Delgado
65080-040 - São Luís - MA – Brasil

LUIZ RUIFENG MEI

MATHEUS SANTOS MARQUES

NATALIA PEREIRA SILVA

TRABALHO REFERENTE À 1ª AVALIAÇÃO - TURMA 01

Resolução das Questões 1 e 3

São Luís – MA
2026

Sumário

1. Introdução 3
2. Metodologia 3
3. Desenvolvimento das soluções 3
3.1 Solução da Questão 1 3
3.2 Solução da Questão 3 5
4. Conclusão 7
5. Referências 8

1. Introdução

Este trabalho constitui a primeira avaliação da disciplina de Introdução à Arquitetura de Computadores. O objetivo principal é aplicar os conceitos teóricos de microcontroladores através do desenvolvimento, compilação e simulação de sistemas baseados no chip PIC16F84A. Para cumprir a proposta, foram selecionadas duas tarefas práticas (Questões 1 e 3) que envolvem o controle de portas de entrada e saída (I/O), a geração de sinais temporizados e a implementação de lógicas combinacionais e sequenciais em linguagem C.

2. Metodologia

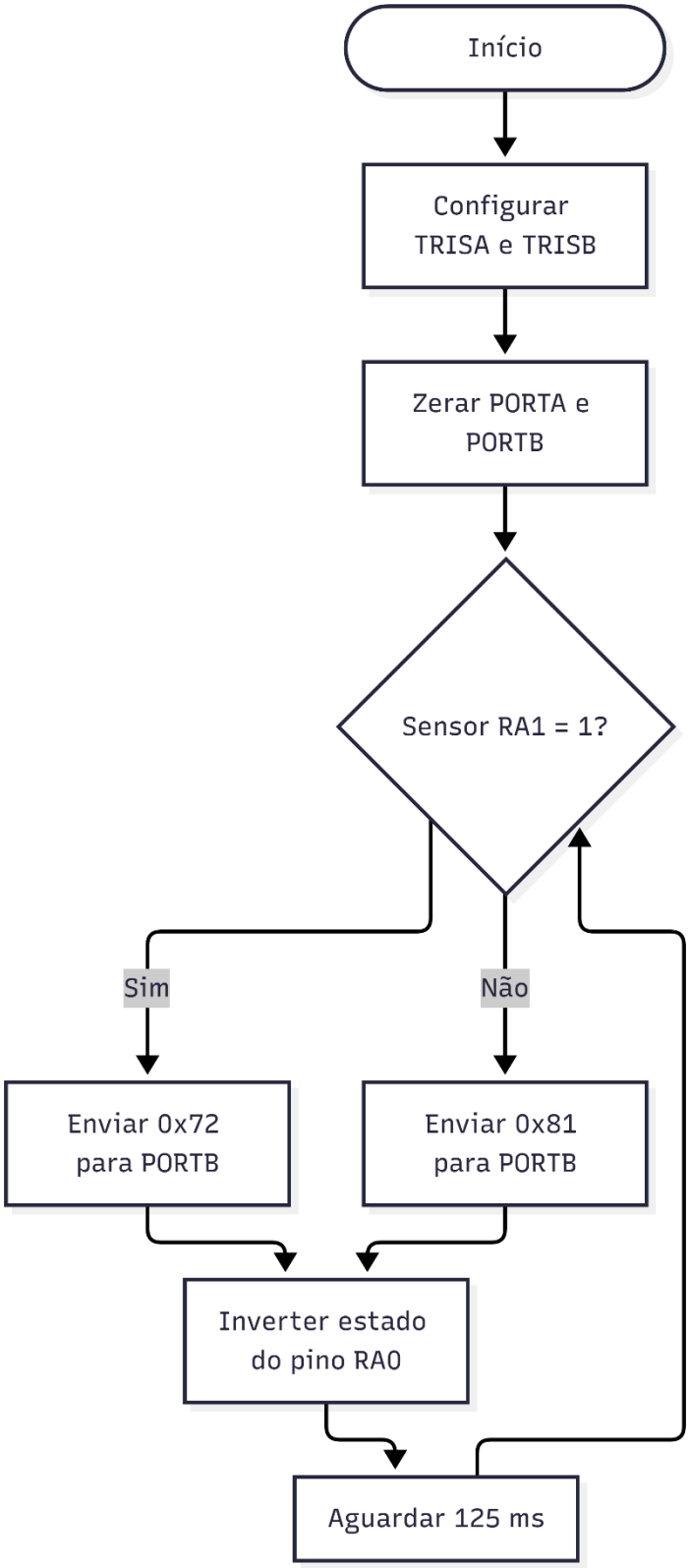
O desenvolvimento prático baseou-se nas ferramentas exigidas para a disciplina. O código-fonte de cada solução foi escrito em linguagem C utilizando o ambiente de desenvolvimento integrado (IDE) MPLAB X em conjunto com o compilador XC8 da Microchip. Após a etapa de compilação (geração do arquivo .hex), os testes e a validação do hardware virtual foram conduzidos no simulador PICsimlab. Em ambas as simulações, o microcontrolador PIC16F84A foi configurado para operar com um clock de 4 MHz, utilizando o registrador interno para garantir a precisão dos atrasos de tempo (delays) solicitados.

3. Desenvolvimento das soluções

3.1 Solução da Questão 1

Análise do Problema: O sistema exige a leitura contínua do bit RA1. Se ativado (nível lógico 1), a porta B deve receber o valor hexadecimal 0x72; se desativado (nível lógico 0), deve receber 0x81. Paralelamente, o bit RA0 deve emitir um sinal quadrado de 4 Hz. Para atingir 4 Hz, o período total da onda é de 250 ms, exigindo que o estado do pino seja invertido a cada 125 ms.

Fluxograma:


Início
Configurar TRISA e TRISB
Zerar PORTA e PORTB
Sensor RA1 = 1?
Sim
Não
Enviar 0x72 para PORTB
Enviar 0x81 para PORTB
Inverter estado do pino RA0
Aguardar 125 ms

Código em C:

```c
// Configurações do Microcontrolador
#pragma config FOSC = XT  // Oscilador a Cristal (necessário para os 4MHz)
#pragma config WDTE = OFF // Desativa o Watchdog Timer (evita resets indesejados)
#pragma config PWRTE = ON // Ativa o Power-up Timer
#pragma config CP = OFF   // Desativa a proteção de código

#define _XTAL_FREQ 4000000 // Clock de 4 MHz exigido
#include <pic16f84a.h>
#include <xc.h>
#include <stdio.h>
#include <stdlib.h>

void main(void) {
    TRISA = 0b00010; // RA1 configurado como entrada; RA0 como saída
    TRISB = 0x00;    // Toda a Porta B configurada como saída

    // Zera as portas para evitar lixo de memória na inicialização
    PORTA = 0x00;
    PORTB = 0x00;

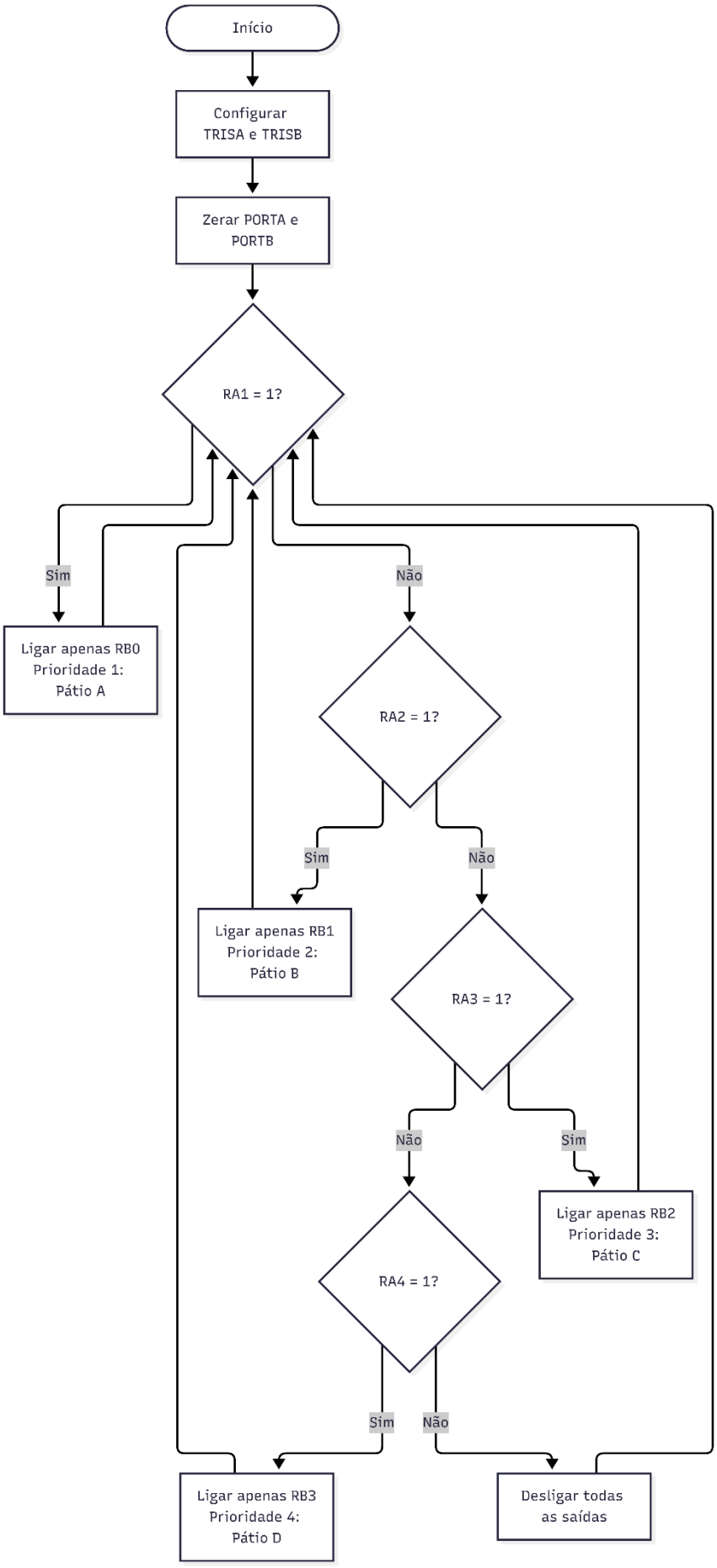
    while(1) {
        // Verifica o sensor em RA1
        if (PORTAbits.RA1 == 1) {
            PORTB = 0x72; // Retorna 0x72 se o sensor for 1
        } else {
            PORTB = 0x81; // Retorna 0x81 caso contrário
        }

        // Gera o sinal quadrado de 4Hz em RA0
        PORTAbits.RA0 = ~PORTAbits.RA0; // Inverte o estado do pino
        __delay_ms(125);                // Atraso de meia onda (125ms + 125ms = 250ms = 4Hz)
    }
}
```

3.2 Solução da Questão 3

Análise do Problema: Trata-se de um sistema de controle de prioridade para esteiras transportadoras vindas de quatro pátios (A, B, C e D), vinculados às entradas RA1, RA2, RA3 e RA4. O Pátio A possui a prioridade máxima (1) e o Pátio D a prioridade mínima (4). A lógica foi implementada através de uma estrutura sequencial if/else if, onde a leitura dos pinos é feita em ordem decrescente de prioridade, garantindo que as saídas RB0 a RB3 correspondam exclusivamente à esteira mais prioritária ativa no momento.

Fluxograma:


Início
Configurar TRISA e TRISB
Zerar PORTA e PORTB
RA1 = 1?
Sim
Não
Ligar apenas RB0 Prioridade 1: Pátio A
RA2 = 1?
Sim
Não
Ligar apenas RB1 Prioridade 2: Pátio B
RA3 = 1?
Não
Sim
Ligar apenas RB2 Prioridade 3: Pátio C
RA4 = 1?
Sim
Não
Ligar apenas RB3 Prioridade 4: Pátio D
Desligar todas as saídas

Código em C:

```c
// Configurações do Microcontrolador
#pragma config FOSC = XT  // Oscilador a Cristal (necessário para os 4MHz)
#pragma config WDTE = OFF // Desativa o Watchdog Timer (evita resets indesejados)
#pragma config PWRTE = ON // Ativa o Power-up Timer
#pragma config CP = OFF   // Desativa a proteção de código

#include <pic16f84a.h>
#include <xc.h>
#include <stdio.h>
#include <stdlib.h>

void main(void) {
    // RA1, RA2, RA3 e RA4 como entrada (1 = entrada, 0 = saída)
    TRISA = 0b11110;
    TRISB = 0x00;    // Toda a Porta B configurada como saída

    // Zera as portas na inicialização
    PORTA = 0x00;
    PORTB = 0x00;

    while(1) {
        // Avaliação em ordem de prioridade
        if (PORTAbits.RA1 == 1) {
            PORTB = 0b00000001; // Pátio A (Prioridade 1): Liga apenas RB0
        } else if (PORTAbits.RA2 == 1) {
            PORTB = 0b00000010; // Pátio B (Prioridade 2): Liga apenas RB1
        } else if (PORTAbits.RA3 == 1) {
            PORTB = 0b00000100; // Pátio C (Prioridade 3): Liga apenas RB2
        } else if (PORTAbits.RA4 == 1) {
            PORTB = 0b00001000; // Pátio D (Prioridade 4): Liga apenas RB3
        } else {
            PORTB = 0x00;       // Se nenhum pátio enviar sinal, apaga todas as saídas
        }
    }
}
```

4. Conclusão

Os testes práticos confirmaram a eficácia do código gerado para o microcontrolador PIC16F84A. Na Questão 1, o sistema conseguiu ler as variáveis de entrada de forma contínua, atuando corretamente sobre os LEDs correspondentes aos sinais 0x72 e 0x81, sem interferir na modulação do sinal de 4 Hz no pino ao lado. Na Questão 3, o arranjo lógico do tipo cascata comprovou ser uma abordagem estável e precisa para resolver problemas de colisão e codificação de prioridades físicas. Em ambos os cenários, a configuração correta dos bits foi determinante para que os códigos funcionassem corretamente. Além disso, a sincronização correta do clock na primeira questão foi fundamental para a estabilidade visual durante a emulação no PICsimlab.

5. Referências

MICROCHIP TECHNOLOGY INC. PIC16F84A Data Sheet: 18-pin Enhanced FLASH/EEPROM 8-bit Microcontroller. Chandler, AZ: Microchip Technology Inc., 2001.

MICROCHIP TECHNOLOGY INC. MPLAB® XC8 C Compiler User's Guide. Chandler, AZ: Microchip Technology Inc., 2020.

PICSIMLAB. PICsimlab - Programmable IC Simulator Laboratory. Disponível em: https://lcgamboa.github.io. Acesso em: 08 out. 2026.